



ESCUELA PROFESIONAL DE INGENIERÍA DE COMPUTACIÓN Y SISTEMAS

INFORME DE CONFIGURACIÓN DE RENDER

**PROYECTO:
AGROSMART INSIGHTS – OPTIMIZACIÓN DE MERCADOS Y PRECIOS
AGRÍCOLAS**

**CURSO:
TALLER DE PROYECTOS**

**PROFESOR:
ING. NORMA VIRGINIA LEÓN LESCANO**

**INTEGRANTE:
León Cangalaya, Gabriel Emilio
Líder DevSecOps – Célula 2**

FECHA: 25/08/2026

LIMA – PERÚ

INFORME DE CONFIGURACIÓN DE RENDER – AGROSMART INSIGHTS

1. Objetivo

El propósito de este informe es documentar la arquitectura, la especificación de infraestructura como código (IaC), el procedimiento de despliegue coordinado, el diagnóstico exhaustivo de fallos y la resolución de incidencias en la plataforma Cloud Render durante el Sprint 1. Se detalla el aprovisionamiento de la base de datos PostgreSQL y del motor de automatización n8n mediante `render.yaml`, los ajustes de rendimiento de memoria, la configuración de reglas de red y la ejecución de migraciones con 9 445 registros de prueba.

1.1. Fundamento metodológico

Render ofrece una plataforma en la nube basada en infraestructura declarativa mediante especificaciones Blueprint (`render.yaml`). Este enfoque permite versionar la infraestructura junto con el código fuente, garantizando entornos reproducibles, trazabilidad de cambios y mitigación de errores de configuración manual entre ambientes (Render, 2026).

En apego a las directrices DevSecOps, la capa backend interconecta el orquestador n8n y la base de datos relacional PostgreSQL a través de una red privada virtual (Private Network), bloqueando accesos no autorizados desde internet y gestionando las credenciales y llaves de modelos de lenguaje mediante variables seguras inyectadas en tiempo de despliegue (NIST, 2022).

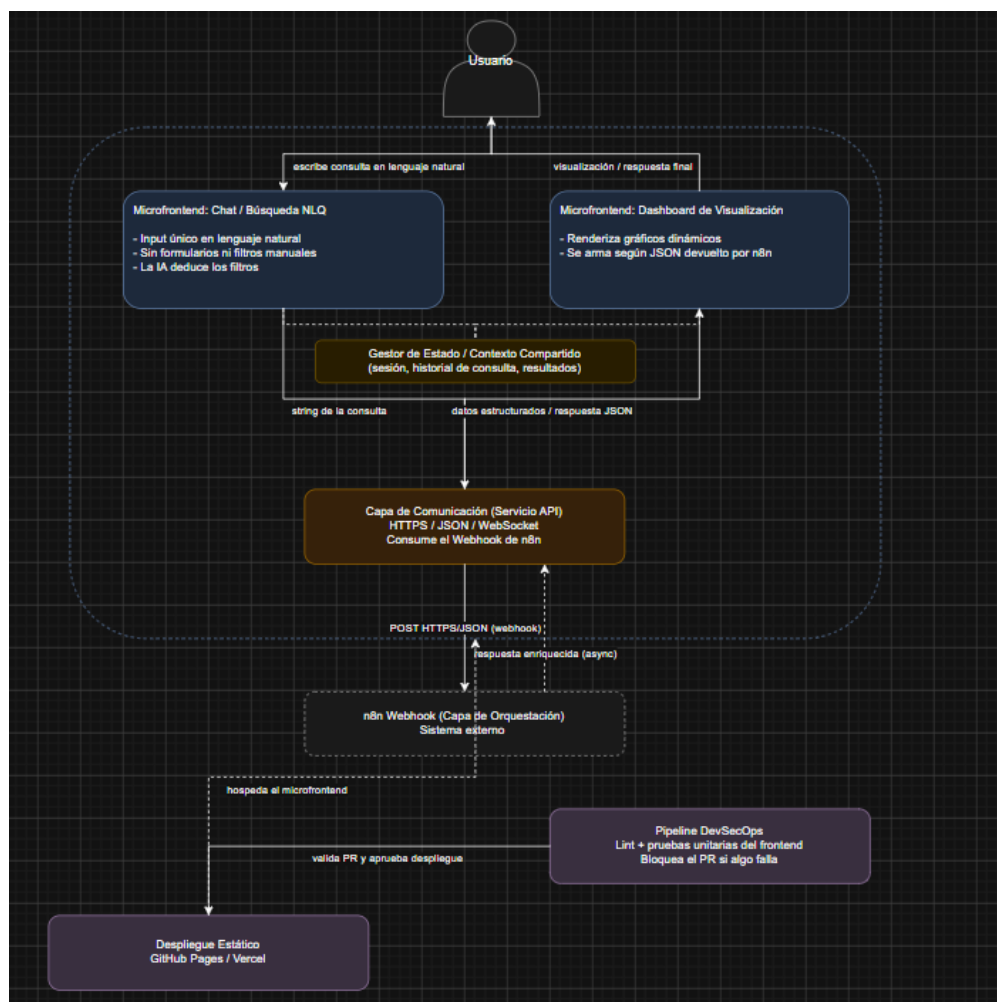
1.2. Arquitectura de Backend en Render

El backend de AgroSmart Insights está compuesto por dos servicios fundamentales:

- agrosmart-postgres: Instancia gestionada de PostgreSQL 18 con la extensión pgvector habilitada, destinada al almacenamiento de catálogos maestros de productos, registros de mercados, series de precios históricos del GMML y vectores de similitud semántica para el agente RAG.
- agrosmart-n8n: Servidor de flujos de trabajo basado en Node.js, desplegado con la versión optimizada n8n:1.83.2, responsable de la recepción de consultas en lenguaje natural, invocación de modelos LLM (Groq y Gemini) y generación de respuestas estructuradas.

Figura 1

Diagrama de arquitectura del backend DevSecOps en Render



Nota. Diagrama que representa la topología interna en Render. El servicio web n8n y la base de datos PostgreSQL se comunican a través de la red privada interna con aislamiento de credenciales y exposición controlada de endpoints webhook. *Fuente:* Elaboración propia a partir del diseño de arquitectura de AgroSmart Insights (Equipo 2 AgroSmart Insights, 2026).

2. Especificación de Infraestructura como Código (Blueprint)

2.1. Estructura del archivo render.yaml

Se diseñó el archivo render.yaml en la raíz del repositorio para definir la infraestructura de manera reproducible. La configuración incluye:

- Servicio de Base de Datos (databases): Nombre agrosmart-postgres, base de datos agrosmart, usuario agrosmart, plan free.

- Servicio Web de Aplicación (services): Tipo web, entorno de contenedor Docker, imagen `docker.io/n8nio/n8n:1.83.2`, puerto 5678 y disco de almacenamiento persistente de 1 GB montado en `/home/node/.n8n`.

2.2. Parametrización y variables de entorno del Blueprint

Se establecieron variables de conexión asociadas automáticamente mediante la cláusula `fromDatabase` (`DB_POSTGRESDB_HOST`, `DB_POSTGRESDB_USER`, `DB_POSTGRESDB_PASSWORD`, `DB_POSTGRESDB_DATABASE`), junto con variables marcadas `sync: false` para la captura segura de API keys en la consola de Render.

Figura 2

Conexión del repositorio en Render para inicializar el Blueprint

You are deploying from a Blueprint for `equipo2-AgroSmart-Insights/Agrosmart-Insights`

Blueprint Name
A unique name for your Blueprint:

Branch
The repository branch with the `(render: yarn)` file:

Blueprint Path (Optional)
The path to the Blueprint file in your repository, `(file://render: yarn)`. Defaults to `(render: yarn)` at the root of your repository:

Review Blueprint configurations from `equipo2-AgroSmart-Insights/Agrosmart-Insights`.

ⓘ All future updates to your `(Blueprint)` file will be synced automatically, which may change your code.

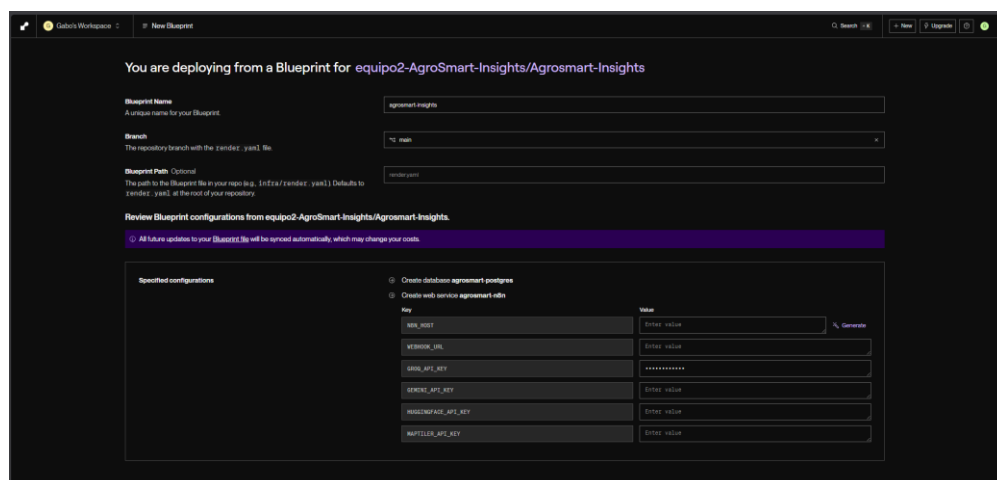
Specified configurations

Key	Value
<code>DB_HOST</code>	*****
<code>DB_NAME</code>	*****
<code>DB_USER</code>	*****
<code>DB_PASS</code>	*****
<code>DB_SCHEMA</code>	*****
<code>DB_DATABASE</code>	*****
<code>DB_PASSWORD</code>	*****
<code>DB_USERNAME</code>	*****

Nota. Captura de pantalla de la consola de Render en el módulo Blueprints, mostrando la vinculación con el repositorio `equipo2-AgroSmart-Insights/Agrosmart-Insights`. *Fuente:* Elaboración propia a partir de la consola de Render (Render, 2026).

Figura 3

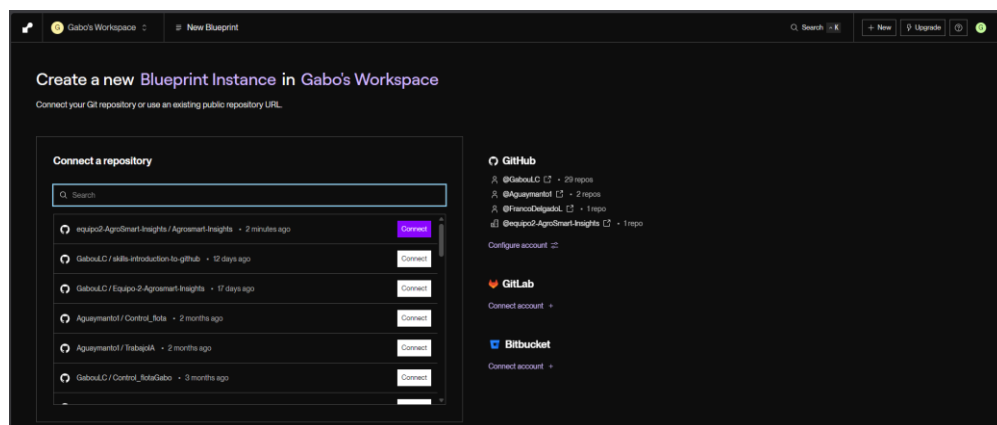
Formulario de configuración de variables de entorno y API Keys en Render Blueprint



Nota. Captura de pantalla de la interfaz de Render mostrando los campos de variables de entorno sensibles (GROQ_API_KEY, GEMINI_API_KEY, HUGGINGFACE_API_KEY, MAPTILER_API_KEY) debidamente completados y protegidos. *Fuente:* Elaboración propia a partir de la consola de Render (Render, 2026).

Figura 4

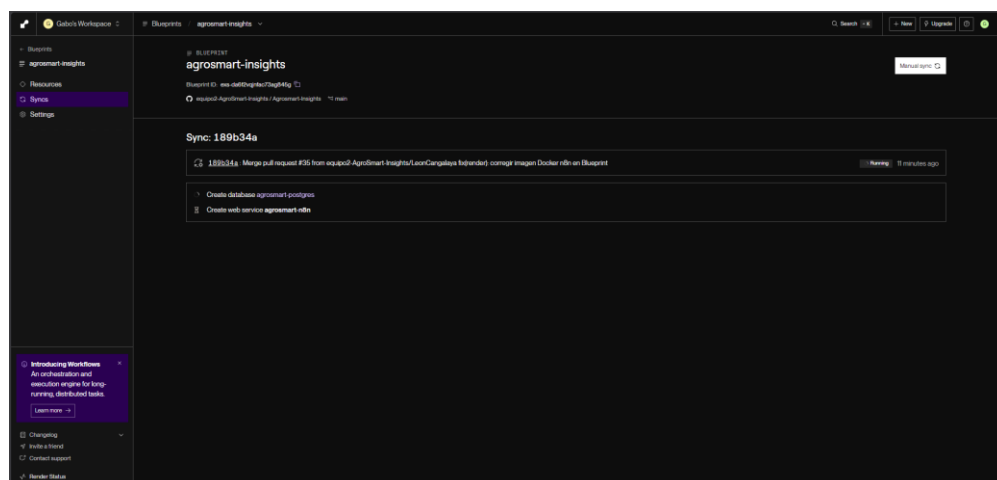
Confirmación de Blueprint y creación coordinada de recursos en Render



Nota. Captura de pantalla del resumen de recursos definidos en render.yaml (agrosmart-postgres y agrosmart-n8n) previo al inicio del aprovisionamiento. *Fuente:* Elaboración propia a partir de la consola de Render (Render, 2026).

Figura 5

Estado de sincronización Running durante el aprovisionamiento de recursos



Nota. Captura de pantalla de la consola de Render reflejando la fase activa de sincronización y despliegue concurrente de la base de datos y el servicio web. *Fuente:* Elaboración propia a partir de la consola de Render (Render, 2026).

3. Diagnóstico de Incidencias Técnicas y Optimizaciones

3.1. Corrección del registro de imagen Docker (PR #35)

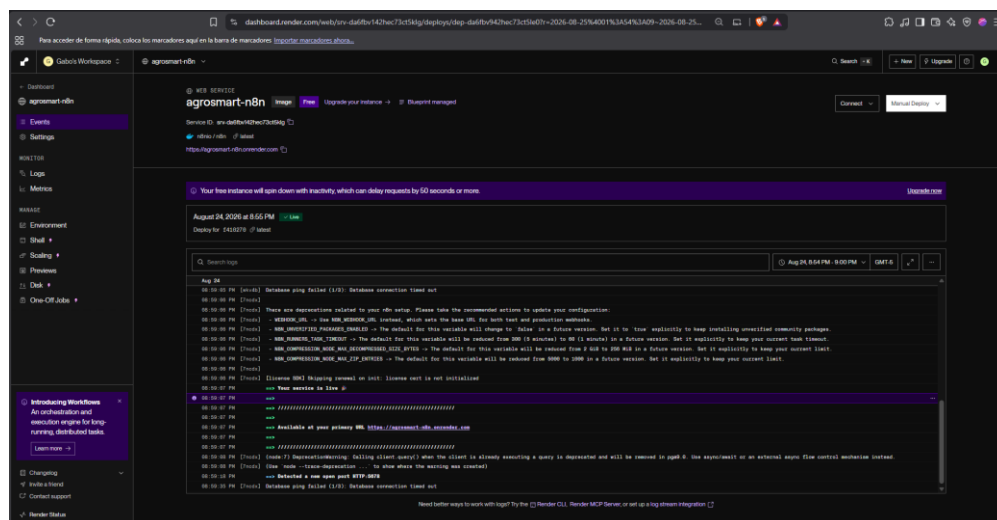
Durante el primer intento de despliegue, Render emitió el error 'the provided URL could not be fetched' al intentar resolver el dominio `docker.n8n.io/n8nio/n8n:latest`. Se solventó mediante el Pull Request #35, reorientando la ruta hacia el repositorio oficial en Docker Hub (`docker.io/n8nio/n8n:1.83.2`) e integrando la directiva `N8N_ENCRYPTION_KEY` con generación automática de clave.

3.2. Optimización de memoria ante fallo OOM (Heap Crash)

Debido a que el tier gratuito de Render cuenta con 512 MB de memoria RAM compartida, n8n sufrió un colapso de memoria durante el arranque (FATAL ERROR: JavaScript heap out of memory, exit code 134), provocando que la aplicación respondiera con Not Found y entrara en un ciclo de reinicios constantes.

Figura 6

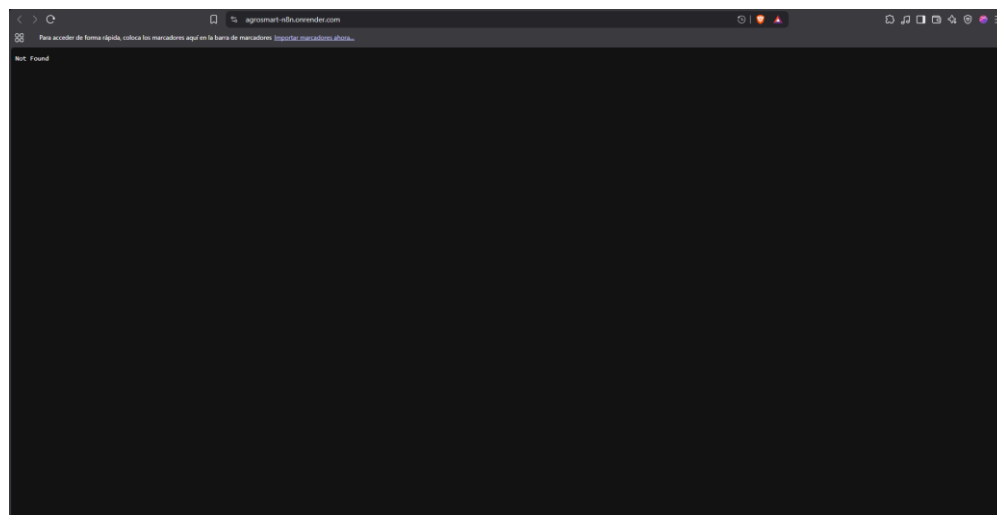
Logs de arranque de n8n registrando inicialización en puerto 5678



Nota. Captura de pantalla del registro de eventos de Render mostrando el mensaje n8n ready on port 5678 previo a la saturación de memoria en versiones no optimizadas. *Fuente:* Elaboración propia a partir de los registros de Render (Render, 2026).

Figura 7

Respuesta Not Found en navegador por caída de servicio previa a estabilización



Nota. Captura de pantalla del navegador web reflejando la indisponibilidad temporal del servicio antes de aplicar las restricciones de consumo de memoria. *Fuente:* Elaboración propia a partir de la navegación web.

Para estabilizar el consumo de recursos se aplicaron las siguientes variables de entorno:

- **NODE_OPTIONS:** --max-old-space-size=384 (fuerza la recolección periódica de basura en Node.js antes de sobrepasar el tope del contenedor de 512 MB).
- **N8N_RUNNERS_ENABLED:** false (desactiva subprocessos concurrentes y procesadores auxiliares en segundo plano).
- **DB_POSTGRESDB_SSL_ENABLED:** false (elimina la sobrecarga de negociación SSL en enlaces sobre la red privada interna).
- **PORT:** 5678 y **N8N_EDITOR_BASE_URL:** <https://agrosmart-n8n.onrender.com> (alineación explícita de puertos y URLs públicas).

3.3. Restablecimiento del esquema de base de datos (Error column User.role does not exist)

Las interrupciones abruptas de n8n por falta de memoria provocaron que las migraciones internas de TypeORM quedaran en un estado corrupto e incompleto. Al acceder a la pantalla /setup para crear el usuario principal, el sistema arrojaba el error column User.role does not exist.

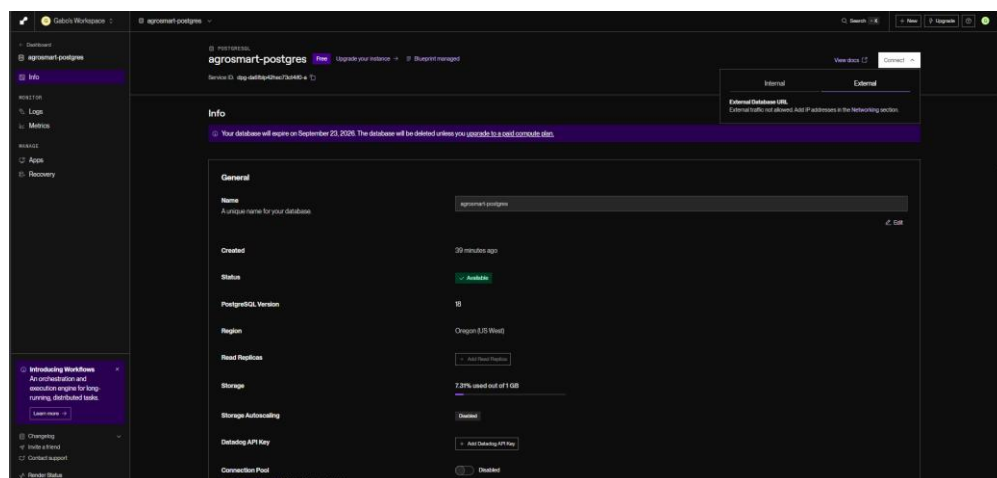
Figura 8

Panel de conexión interna y externa de agrosmart-postgres

Nota. Captura de pantalla de la base de datos agrosmart-postgres en estado Available, indicando credenciales y parámetros de conexión interna y externa. *Fuente:* Elaboración propia a partir de la consola de Render (Render, 2026).

Figura 9

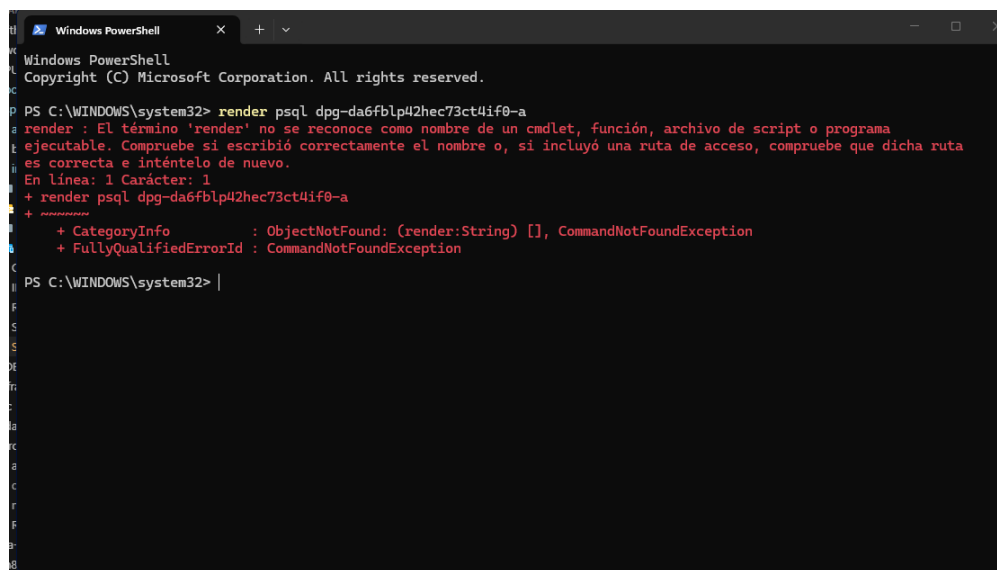
Configuración de reglas de firewall IP en PostgreSQL Inbound Rules



Nota. Captura de pantalla de la sección Networking de agrosmart-postgres permitiendo la conexión segura desde la dirección IP de la estación de trabajo local. *Fuente:* Elaboración propia a partir de la consola de Render (Render, 2026).

Figura 10

Error en asistente de creación de cuenta Owner: column User.role does not exist



Nota. Captura de pantalla del formulario de registro de n8n mostrando la notificación de error en base de datos debido al esquema corrupto. *Fuente:* Elaboración propia a partir de la interfaz de n8n.

El procedimiento de saneamiento consistió en: (a) habilitar temporalmente la IP local en las reglas de entrada de PostgreSQL, (b) conectarse mediante psql y ejecutar

DROP SCHEMA public CASCADE; CREATE SCHEMA public; GRANT ALL ON SCHEMA public TO agrosmart;, y (c) ejecutar un Manual Deploy en n8n para que recreara limpiamente sus 25 migraciones estructurales sobre PostgreSQL 18.

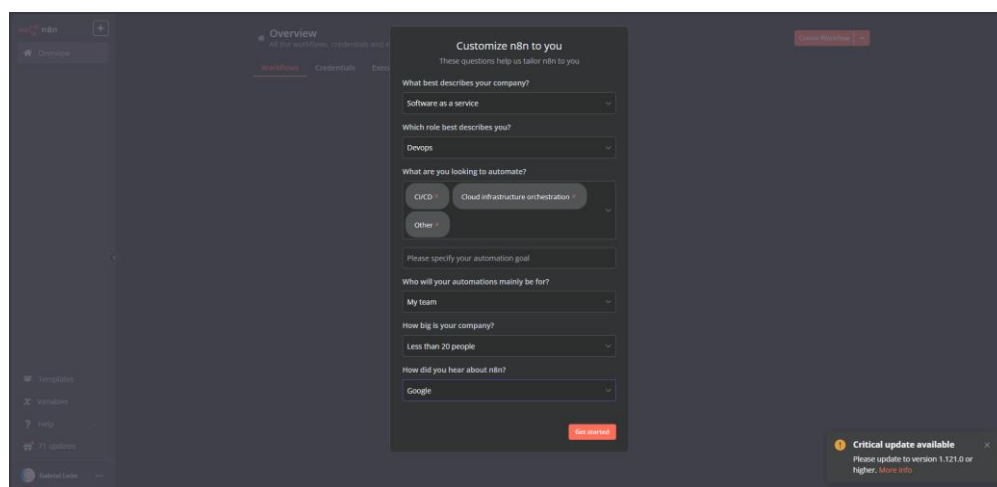
4. Validación Operativa, Migraciones y Carga Masiva de Datos

4.1. Acceso a n8n y registro de cuenta Owner

El servicio n8n arrancó de manera impecable en <https://agrosmart-n8n.onrender.com>. Se completó el asistente de incorporación creando la cuenta de propietario a nombre de Gabriel Emilio León Cangalaya.

Figura 11

Panel principal de n8n operativo con cuenta Owner autenticada



Nota. Captura de pantalla del panel principal de n8n confirmando la sesión activa del usuario Gabriel León y el espacio de trabajo preparado para flujos analíticos. *Fuente:* Elaboración propia a partir de la interfaz de n8n.

4.2. Ejecución de migraciones de negocio y carga masiva SQL

Desde la consola PowerShell local y haciendo uso del cliente psql sobre la URL externa de agrosmart-postgres, se ejecutaron exitosamente los siguientes scripts:

- 001_schema.sql: Creación de la extensión vector, tablas de catálogo de productos, mercados, precios históricos y bitácora de auditoría.
- 002_indexes.sql: Generación de índices btree y compuestos para acelerar búsquedas analíticas por fechas y entidades.
- 002_seed_v2.sql: Inserción masiva de 9 445 registros oficiales de precios históricos del Gran Mercado Mayorista de Lima (GMMML) mediante el comando COPY.

Figura 12

Ejecución exitosa de migraciones y seed de 9 445 filas en PostgreSQL Render

Nota. Captura de pantalla de la terminal psql con las respuestas CREATE EXTENSION, CREATE TABLE, CREATE INDEX y COPY 9445 confirmando la persistencia de datos. *Fuente:* Elaboración propia a partir de la consola de psql.

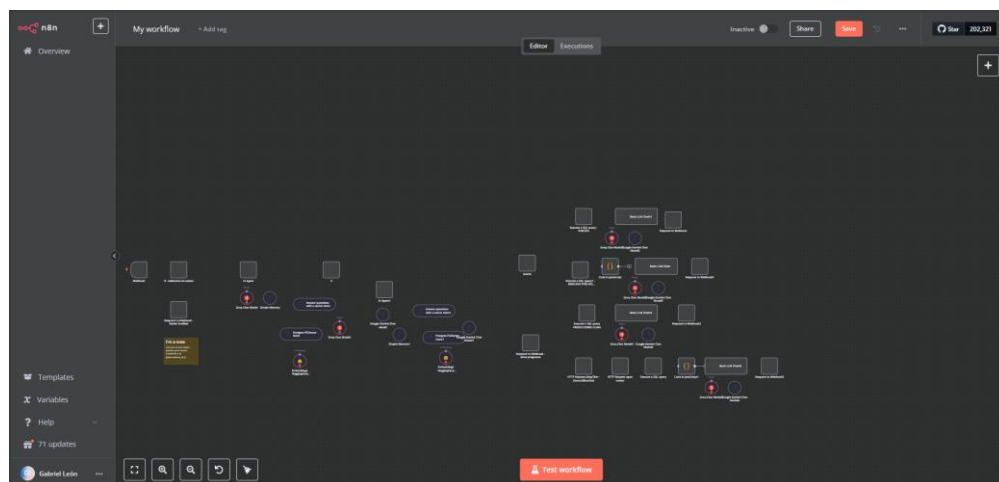
4.3. Importación, compatibilidad y activación del flujo WF2

Se importó en n8n el archivo oficial wf2-api-chat-analisis-predictivo.json. La primera importación reveló incompatibilidad de typeVersion: el JSON provenía de una versión más reciente que n8n 1.83.2 (Webhook 2.1, Gemini 1.1, AI Agent 3.1, Switch 3.4). n8n no reconocía esos nodos como ejecutables y al activar el flujo devolvía Cannot read properties of undefined (reading 'execute').

Se normalizaron las versiones internas al conjunto soportado por 1.83.2, se completó el campo Data Name (informacion_agricola) en las herramientas de vector store y se sustituyó la clave de MapTiler embebida en el JSON por la expresión {{ \$env.MAPTILER_API_KEY }}, en cumplimiento del principio de cero secretos en código.

Figura 13

Workflow WF2 importado en el editor visual de n8n



Nota. Captura del lienzo de n8n con el grafo de nodos del flujo WF2 tras la importación inicial. *Fuente:* Elaboración propia a partir de la interfaz de n8n.

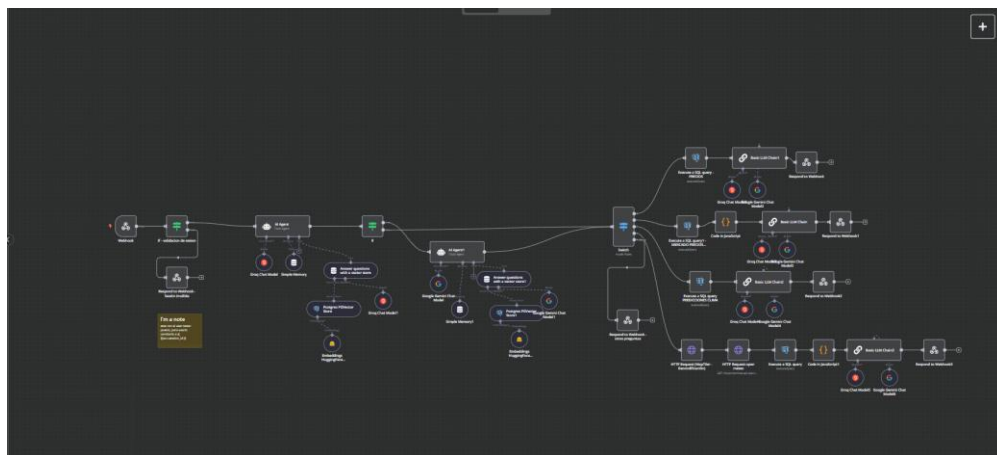
Figura 14

Campo Data Name requerido en la herramienta Vector Store de WF2

Nota. Captura del nodo Answer questions with a vector store. El campo Data Name es obligatorio en typeVersion 1 y se completó con informacion_agricola. *Fuente:* Elaboración propia a partir de la interfaz de n8n.

Figura 15

Flujo WF2 operativo en n8n cloud con Groq, Gemini, RAG y ramas SQL



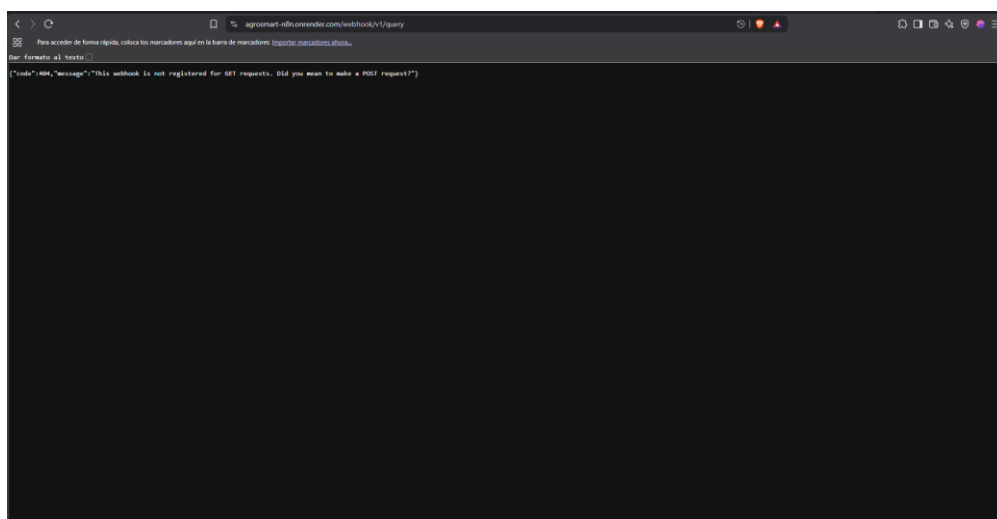
Nota. Captura del lienzo completo de WF2 tras la corrección de typeVersion y el enlace de credenciales Postgres, Groq, Gemini y Hugging Face. *Fuente:* Elaboración propia a partir de la interfaz de n8n.

4.4. Verificación pública del webhook de producción

La URL de producción del webhook es <https://agrosmart-n8n.onrender.com/webhook/v1/query>. Una petición GET desde el navegador devolvió el JSON 404 «This webhook is not registered for GET requests. Did you mean to make a POST request?». Esa respuesta demuestra que el disparador está registrado y activo: n8n rechaza GET porque el nodo Webhook está configurado en POST.

Figura 16

Respuesta del webhook de producción ante una petición GET del navegador



Nota. Captura del navegador en <https://agrosmart-n8n.onrender.com/webhook/v1/query>. El código 404 aclara que el endpoint existe y espera POST, que es el método usado por n8nClient.js. *Fuente:* Elaboración propia a partir de la navegación web.

5. Conclusión

La infraestructura de backend en Render quedó desplegada, optimizada y operativa. Se superaron los límites de memoria del plan gratuito, se saneó el esquema interno de n8n, se cargaron 9 445 registros del GMLL, se compatibilizó WF2 con n8n 1.83.2 y se verificó el webhook público. El backend cloud del Sprint 1 queda listo para atender al frontend en Vercel.

Referencias

- Equipo 2 AgroSmart Insights. (2026). Agrosmart-Insights [Repositorio de software]. GitHub. <https://github.com/equipo2-AgroSmart-Insights/Agrosmart-Insights>
- National Institute of Standards and Technology. (2022). Secure Software Development Framework (SSDF) Version 1.1 (NIST SP 800-218). <https://csrc.nist.gov/Projects/ssdf>
- n8n.io. (2026). Hosting n8n with PostgreSQL. Documentation. <https://docs.n8n.io/hosting/databases/postgresql/>
- Red Hat. (s. f.). ¿Qué es DevSecOps? <https://www.redhat.com/es/topics/devops/what-is-devsecops>
- Render. (2026). Blueprint specification. <https://render.com/docs/blueprint-spec>
- Render. (2026). Deploy n8n on Render. <https://render.com/docs/deploy-n8n>
- Render. (2026). PostgreSQL on Render. <https://render.com/docs/databases>